

QUALITY SCORECARD FRAMEWORK

A measurable, gate-based approach to building quality into every unit of work, with the capability and competency model that explains the results

Item	Details
Document type	Quality framework
Version	1.1 - Updated with ODTQRC tool notes
Scope	Every unit of work (task, activity, milestone, project), technical and non-technical, for individuals, teams and the organisation
Source	Framework discussion led by Subramanian S, 21 September 2026; ODTQRC and audit-tool notes (Arpan Kumar); drafting additions marked as proposed or illustrative (notably Section 16)
Status	Draft for review; internal adoption first

Version 1.1 consolidates the ODTQRC task definition, the Quality Scorecard, the QTC rule and the Capability & Competency model into a single framework. Illustrative examples are labelled as such; IPPPA and the predictive and prescriptive analytics layers are planned extensions.

Contents

1. Purpose
2. Background: why quality has to be measured
3. Scope and levels of application
4. Core principles
5. The framework at a glance
6. The scoring scale, colour bands and targets
7. Task definition: the ODTQRC template
8. The seven-step scorecard design method
9. The expert judgement system: guidelines
10. Quality gates and the QTC rule
11. Self-appraisal
12. Diagnosing reds
13. Aptitude, Capability and Competency
14. Tool and product vision
15. Reference scorecard library
16. Pilot, roadmap and link to the Data-Driven Development Framework
17. Framework evolution
18. Considerations and open points

1. Purpose

This document defines a repeatable framework for making the quality of any piece of work explicit, measurable and enforceable before the work moves on. It applies to engineering deliverables and to everyday work alike: a requirements document, a cloud deployment, a client email, a meeting, a brainstorming session or a prompt.

Before an activity starts, the person doing it states the purpose, agrees the quality KPIs and the target score, and knows how the work will be judged. When the activity is complete, it is scored against written guidelines, shown as a Red / Amber / Green (RAG) result, and allowed to proceed only if it reaches the target.

The same framework also answers the next question. When work comes back red, is it a skill gap, an aptitude mismatch or a will problem? The Aptitude, Capability and Competency model in Section 13 provides that diagnosis and the route to fix it.

Core sequence: ODTQRC task definition (Objectives -> Deliverables -> Timeline -> Quality KPIs -> Risks -> Closure) -> Guidelines -> Target -> Self-appraise (can I deliver this?) -> Do the work -> Judge -> Gate (QTC) -> Diagnose -> Learn.

2. Background: why quality has to be measured

As teams scale to hundreds of people across dozens of products, the organisation can no longer rely on reading every document or trusting self-declared status. At 300 people each performing around eight types of task, roughly 2,400 tasks a day pass through the system. Nobody can review that by hand, and nobody should have to. Leadership should only need to look at the reds and greens, and trust that a green is really green.

Approach	What typically happens	Limitation / Learning
1. Build first	Engineers listen for ten minutes, then start coding.	No spec, no purpose; problems surface weeks later as rework and dissatisfaction.
2. Review by reading	Leads and leaders read every spec and deliverable.	Does not scale beyond a handful of people or products.
3. Ask an AI for a rating	"How good is my document?" returns a number.	The number has no agreed KPIs or guidelines behind it, so it is arbitrary and not comparable.
4. Self-declared status	Teams report their own RAG status.	Tends to be all green while outcomes are poor; reporting is not honest.
5. Scorecard-led (this framework)	Purpose, KPIs, guidelines and targets defined before the work; scored and gated after.	Quality is designed in, judged consistently, and visible at any level.

The immediate problem is limited, unstructured and low-quality research and preparation before work begins. Work that starts from weak research inherits its weaknesses, and nobody sees them until much later. Scorecards themselves are not new: the Balanced Scorecard has long been used to rate companies. What is new is applying them to every unit of work, with AI making KPI and guideline design practical at scale.

The honest-RAG lesson is well known from Ford under Alan Mulally. Week after week every business unit reported green while the company was losing billions. Only when the first amber and red appeared, and people saw it was safe to report them, did the problems become visible and fixable. The goal is not a board of greens; it is a board that tells the truth.

Garbage in, garbage out: a badly designed scorecard produces "beautiful rainbow colours" that mean nothing, and within a week the whole system is discarded. Every KPI and guideline must earn its place.

3. Scope and levels of application

A quality scorecard can be built at any level. The same mechanism applies whether the subject is a single task or the whole organisation.

Dimension	Levels	Notes
Unit of work	Task -> Activity -> Milestone -> Project	A milestone is a set of tasks done to achieve a higher-order business objective. A project has milestones; milestones have activities; activities have tasks.

Dimension	Levels	Notes
Who is measured	Individual -> Team -> Organisation	Scores roll up; a project cannot be green if the items beneath it are red.
Type of work	Technical and non-technical	Specs, code, deployments, reports, proposals, client emails, meetings, prompts; "all of them have to deliver green quality".
Adoption	Internal first -> broader teams -> democratised -> external	Used and proven internally before it is offered to anyone else (see Section 17).

4. Core principles

- 1. Start with why.** Every task begins with a short, articulated purpose: one or two statements, not two or three pages. If someone cannot say why they are doing a task, they are not ready to do it. No task starts without objectives.
- 2. Sharpen the saw before cutting.** Preparation, specification and research come before building. Trial-and-error is expensive in people-hours, cloud cost and rework.
- 3. Define quality before the work, not after.** The KPIs, guidelines and target are known upfront, so people can aim at them.
- 4. LLM first, human second.** Ask the AI for candidate KPIs and guidelines first, then apply human judgement: "throw your hat in the ring second".
- 5. Guidelines make judgement repeatable.** Qualitative statements for every score level, then quantitative measures, turn an opinion into an expert judgement system.
- 6. Green must be really green.** Honest reporting matters more than a green board. An early rainbow is healthy; a permanently green board with poor outcomes is not.
- 7. Gate, do not guess.** If the work is below target, it does not move to the next phase; it is redone.
- 8. Quality, time and cost are one AND.** A task is green only if it meets quality, time and cost together (the QTC rule).
- 9. Fit the target to the need.** Do not over-engineer a proof of concept for a 9, and never accept shoddy work at 4 or 5.
- 10. Self-appraisal and self-accountability come first.** People who cannot honestly score their own work cannot be relied on to score anything else. Objectives are self-accountable: the owner answers for them.
- 11. Right person, right task.** Reds are diagnosed, not blamed. Consistent reds usually mean the wrong allocation, and the fix is allocation or training.

5. The framework at a glance

Stage	Key steps	Key output
Define (ODTQRC)	Objectives (SMART); deliverables and scope; timeline with WBS effort estimate; quality KPIs; risks and mitigations; closure and tracking	An agreed, clarity-checked task definition; no start without objectives
Design the scorecard	Discover KPIs with an LLM; define KPIs; write qualitative and quantitative guidelines	A scorecard that any judge can apply consistently
Set target and gate	Choose the target score for the context; agree time and cost	An explicit pass condition before work starts
Self-appraise and execute	Honestly assess whether you can deliver the target by the date and cost; raise gaps; do the work	Realistic commitments and gaps surfaced before work starts
Judge and gate	Expert or LLM judge scores against guidelines; apply QTC	RAG result; proceed or redo
Diagnose and act	For reds: skill, aptitude or will; train, reassign or exit	Better allocation and targeted upskilling
Learn	Refine guidelines; add to reference library; analyse trends	A growing, improving body of scorecards

6. The scoring scale, colour bands and targets

Every KPI and every scorecard is scored on an 11-point scale from 0 to 10. The colours follow a fixed, long-standing convention so that a single glance at a sheet or dashboard shows where attention is needed. Hex values are the suggested implementation of the standard spreadsheet palette described in the discussion.

Score	Band	Meaning	Fill / font (suggested)
10 and 9	Dark green	Excellent. Target for foundational artefacts.	00B050 / white bold
8	Light green	Good. Default target for most work.	92D050 / black
7	Very light grey	Acceptable. Minimum floor for day-to-day work.	D9D9D9 / black
6	Amber	Below standard; acceptable only where a POC target of 6 is set.	FFC000 / black
5	Dark amber (orange)	Average; not acceptable, even for a POC.	ED7D31 / white
4	Red	Poor.	FF0000 / white
3, 2, 1 and 0	Dark red	Unacceptable. All four are treated the same.	C00000 / white

6.1 Setting the target

Every scorecard carries a target column next to the score. The target is set for the context of the work, not fixed globally.

Context	Target	Rationale
Foundational artefacts (e.g. requirements or specs that a whole team builds on)	10	An error here multiplies: eight people working for two weeks is 640 hours built on one document.
Market-ready product	9	A product going to market is built to 9; this is why only a few products can be taken to market at a time.
Standard work and deliverables, including product builds by new engineers	8	The default. A good organisation creates tasks and staffs them with people who can deliver greens.
Organisational floor ("worst case")	7	Even a routine report or client email should not fall below 7.
Proof of concept / formative product	6 to 7	Trial-and-error learning must be sustainable, but no shoddy work (no 4s or 5s). A formative internal product may be set at 6; the general POC expectation is 7.

Targets also interact with complexity. As work gets more complex, fewer people can deliver an 8. Sustaining 8 as complexity rises is what makes an organisation exceptional.

Design principle: If the need is 6, do not over-engineer for 8. Where an error multiplies downstream, set the target high.

7. Task definition: the ODTQRC template

Every task is defined with ODTQRC before it starts. The template addresses the core problem directly: research and preparation that is limited, unstructured or low quality. It turns "I got it" into a written definition that can be checked, scored and tracked. No task starts without objectives.

Element	Meaning	What good looks like
O - Outcome / objectives	Why the task exists and what result it must produce	SMART objectives: number-based, time-bound and self-accountable
D - Deliverables / scope	What will be produced, and what is in and out of scope	A bounded list of deliverables
T - Timeline / implementation plan	When and how the work will be done	Plan built from a work breakdown structure (WBS), with effort estimated per item, milestones and dates

Element	Meaning	What good looks like
Q - Quality KPIs	How the quality of the deliverables will be judged	A scorecard with KPIs, guidelines and a target (Sections 8 and 9)
R - Risks and mitigations	What could stop the task meeting its objectives	Named risks, each with a mitigation and an owner
C - Closure and tracking	How progress is tracked and how the task is closed	Tracking cadence, closure criteria, and the QTC result recorded at closure

7.1 SMART objectives

- **Number-based:** objectives state measurable targets, not intentions.
- **Time-bound:** every objective has a date; there are no "timeless" tasks.
- **Self-accountable:** the person who owns the task answers for the objective.
- **No start without objectives:** a task with no agreed objectives does not begin.

7.2 How ODTQRC connects to the rest of the framework

O and D are the purpose and scope steps of scorecard design (Section 8). Q points to the scorecard itself. T supplies the Time target for QTC, and the WBS effort estimate multiplied by people's time, plus infrastructure, supplies the Cost target. C is where tracking happens and where the QTC result is recorded when the task closes.

Two different Cs: in ODTQRC, C stands for Closure and tracking. In the QTC rule (Section 10), C stands for Cost. The two are kept distinct.

7.3 Clarity agent

A clarity agent reviews each ODTQRC before it is accepted. It probes what is missing or ambiguous, such as unstated objectives, undefined deliverables or unestimated effort, so gaps surface before the work starts rather than weeks later. The detailed behaviour of the agent is proposed and to be confirmed.

8. The seven-step scorecard design method

Scorecards are designed, not improvised. The seven steps below turn a vague sense of "good" into something that can be scored the same way by any competent judge. Before any activity, the person doing it defines the purpose, scope and quality KPIs, starting from a reference scorecard where one exists (step 7) and adapting it to the task.

8.1 Articulate the purpose

Write why the task exists. The tool should require it and it should be audited. Purpose is short, one or two statements, and must be articulated, not just "known". Articulation is not fluency in English; it is the ability to state the why clearly and precisely.

8.2 Define the scope

List what the task covers, for example "these are the seven things we have to do". Scope bounds the work so that quality can be judged against something specific.

8.3 Discover candidate quality KPIs with an LLM

Ask the LLM first what KPIs would indicate quality for this type of work. Do not lead it with your own answer. Then review, challenge and add to its suggestions.

8.4 Define the quality KPIs

Settle on a small set of KPIs, typically five or six, that together describe quality for this type of work. Each KPI must be observable in the output. If a KPI cannot be judged, it produces noise and should be dropped or redefined.

8.5 Write qualitative guidelines

For every KPI, write an English statement describing what each score from 0 to 10 looks like. That is 11 statements per KPI, about 66 for a six-KPI scorecard. Some guidelines can be composite, covering more than one aspect. For example, 0 might be "very low clarity, cumbersome, no evident purpose". These statements are what turn a rating into an expert judgement.

8.6 Write quantitative guidelines

Replace adjectives with measures wherever possible. "A beautiful document" is subjective; counts, percentages, presence or absence of required elements, and thresholds are not. The quantitative layer works like a training manual: this is exactly how to score.

8.7 Build a diverse library of reference scorecards

Build scorecards across many types of work, not just one kind. Over time this grows into a library, and anyone starting a new task can take a reference scorecard and adapt it. People are expected to understand the intent of a reference, not copy it blindly.

Refinement loop: when scores look wrong, the cause is often not the scorecard but unclear guidelines. Next it is people interpreting the guidelines in their own way, which calls for user education. In the end the guidelines are made explicit enough that no interpretation is possible.

9. The expert judgement system: guidelines

A scorecard is only as good as the guidelines that let a judge, human or LLM, apply it. Asking "how good is my document?" produces an unanchored number. Breaking quality into KPIs and giving 11 anchored statements for each lets the judge do a far better job.

Layer	What it contains	Why it matters
Quality KPIs	5 to 6 dimensions of quality for the work type	Breaks "quality" into things that can be judged separately
Qualitative guidelines	An English statement for each score 0 to 10, per KPI	Gives the judge anchors; reduces arbitrary scoring
Quantitative guidelines	Measures and thresholds per score level	Removes interpretation; makes scores reproducible
Scoring manual	How to apply the guidelines, with worked examples	Lets people "hit the target" deliberately, like a training manual

9.1 Worked example (illustrative)

The example below shows the shape of the guidelines for one KPI, "Unambiguity", in a requirements-document scorecard. It is illustrative only; a real scorecard needs all 11 levels for each KPI, validated by people who have written such documents many times.

Score	Qualitative guideline	Quantitative guideline
10	Every requirement has exactly one reasonable reading; independent teams would build the same thing.	100% of requirements have a testable acceptance criterion; zero undefined terms; glossary complete
8	Requirements are clear; a small number of minor clarifications needed.	>= 90% with acceptance criteria; <= 2 undefined terms, none critical
6	Mostly understandable, but several requirements are open to interpretation.	70-89% with acceptance criteria; several undefined terms
4	Frequent vague language; readers disagree on intent.	40-69% with acceptance criteria; key terms undefined
0-3	Requirements are statements of intent, not specifications; no clear purpose.	< 40% with acceptance criteria; no glossary

Other KPIs for the same scorecard might include purpose clarity, scope completeness, testability, traceability and structure (illustrative).

10. Quality gates and the QTC rule

10.1 Gate rules

- No task starts without SMART objectives recorded in its ODTQRC.

- If a piece of work does not reach its target, it does not move to the next phase. It is redone and resubmitted.
- A foundational artefact in dark red, for example a product scope, stops the project at that point. Nothing downstream should start.
- A project is green only when the work beneath it is green. A car is only as good as its smallest correctly fitted part.
- Repeated reds trigger a review of allocation. The discussion suggested three reds as the point at which a person comes off the project; this is proposed and subject to HR policy (see Section 12 for diagnosis).

10.2 The QTC rule

Green has three factors: Quality, Time and Cost. They combine as a logical AND (multiplication), not an average. Each factor is either met (1) or not met (0), and the task result is $Q \times T \times C$.

Factor	Met when	Example
Q - Quality	Scorecard result is at or above the agreed target	Target 8, delivered 8: 1
T - Time	Delivered by the agreed time	Due Monday 4:00 pm, delivered 3:59 pm: 1
C - Cost	Within the agreed cost: company time cost (people-hours) plus infrastructure cost, not just cloud bills	Agreed \$8.00, spent \$8.01: 0
Result	Q AND T AND C (not OR)	1 AND 1 AND 0 = 0, so the task is not green; only 1 AND 1 AND 1 = 1

This is deliberately strict. It is fair because the targets are agreed upfront and progress is visible on a real-time dashboard. If more time or money is needed, it is negotiated before the deadline, not explained after it. Teams own their misses and learn from them.

Current emphasis: the most common reds are expected on quality and time. Cost is tracked but is less critical at this stage. A timeline is set on day one, so there are no "timeless" projects hiding behind sprint after sprint.

11. Self-appraisal

The tool starts with self-appraisal. Before anyone else judges the work, the person scores it against the same scorecard. Self-appraisal is private: it is not shown to others and is not an assessment. Its purpose is self-awareness, so people know where they stand before they commit.

- **Honesty over optimism.** "Yes, I can do anything" is not an answer. The question is whether this person can deliver this green, by this date, at this cost.
- **Raise data, not conclusions.** If there is a gap, the person states the facts and lets the lead decide. Example: "This needs C2 capability, I am at C4, closing the gap takes about 18 hours, and I have 8 free hours in the next four days." The lead might add hours, reassign, or re-scope.
- **Estimate with a WBS.** Effort and time commitments come from breaking the work down (the T in ODTQRC), not from guesswork, so self-appraisal rests on evidence.
- **A leading indicator.** A person who cannot consistently self-appraise will struggle to apply any other scorecard and slows the project down.
- **Team self-appraisal.** Teams can rate themselves the same way on product vision, project scope and product architecture, against aptitude, capability and competency.

12. Diagnosing reds

A red is information, not blame. Blaming is a poor culture; if the wrong person was put on the task, the allocation was wrong. When someone consistently scores red, for example a 4 against a target of 8, there are three possible causes, and each has a different remedy.

Cause	How it shows	Remedy
Skill	Good aptitude but has not learned the technology or method yet (e.g. strong Java engineer on a deep C task).	Train. Skills can be taught; give targeted activities or a short training module before the task.

Cause	How it shows	Remedy
Aptitude	Struggles even after training; lacks the underlying talent for this type of work.	Reassign to tasks that fit their aptitude. Aptitude training is seldom possible and improves only in small increments.
Will	Capable but unwilling to do the work.	Discuss fit openly; if the work is not wanted, help the person find a better-suited role (proposed; subject to HR policy).

If everything is red, the problem is the allocation or the team, not the individuals. The aim is always-green scorecards: the right people on the right tasks.

13. Aptitude, Capability and Competency

To put the right person on the right task, the framework separates three things that are often blurred together. It also separates talent from skill. Hiring rounds such as coding and DSA are checks of talent, not skill.

Term	Definition	How it is gained or shown
Aptitude (talent)	Natural aptitude for a type of work; a "God-given gift".	Tested by aptitude-style rounds; rarely trainable, only slow incremental gains.
Skill	Specific learned know-how: a language, tool or method.	Taught through training, typically weeks rather than months.
Capability	The ability to do something; demonstrated learning.	Built from skills. It can itself be graded (light, normal or dark green) but it is still capability, not competency.
Competency	Demonstrated performance: done successfully, repeatedly, at the required complexity, in the real world.	Earned only by track record: a stated number of successful deliveries of the required complexity.

Progression runs Aptitude -> Skills -> Capability -> Competency. Years of experience are not an indicator: people can spend years doing substandard work.

13.1 Levels (C1 to C6)

Aptitude, capability and competency are each rated on levels C1 to C6, where C1 is the highest. A role or task states the minimum level it needs, for example "minimum C2 aptitude, C2 capability". Detailed descriptors for each level are to be defined (see Section 18).

13.2 Evaluation order

Step	Question	If yes	If no
1	Does the person have the required competency?	Sweet spot: assign.	Go to step 2.
2	Does the person have the required aptitude (e.g. C1 or C2)?	This is the base; go to step 3.	Do not assign; reassign to fit aptitude.
3	Does the person have the required capability level?	Assign; competency will build with delivery.	Go to step 4.
4	Can the capability gap be closed in time?	Upskill first (e.g. a one-week module), then assign.	Raise the data; adjust time, scope or allocation.

Example: someone asked to write a requirements document has no competency (they have not done it many times) but has high aptitude. Their capability is largely LLM-driven, and their original ideation is still developing. Before they write the document, they are given activities that build creativity, critical thinking and business insight to reach the needed level. A week of preparation is cheap; a weak requirements document misleads the tech lead, the architecture and the whole team.

13.3 Competency evidence thresholds (examples from the discussion)

Area	Indicative competency evidence
Requirements documents	Written for 40 to 50 sizable projects, surviving critical customer reviews, consistently correct

Area	Indicative competency evidence
Knowledge graphs (or similar technology)	X successful releases at the required complexity (X may be 1 to 3)
Product scope and vision	Defined at least five times
Designing scorecards	8 to 10 diverse scorecard types, top 3 variants of each done about 3 times, implemented for a team over 2 to 3 years

13.4 Qualitative and quantitative job descriptions

Most job descriptions are qualitative: "should have experience designing neural-network systems". The framework asks for two versions of every JD:

- **Qualitative JD:** the capabilities the role needs, described in words.
- **Quantitative JD:** SMART, countable evidence, e.g. "designed at least three large end-to-end systems", "four to five years on knowledge graphs". The quantitative JD becomes the competency scorecard for the role.

Both apply to capability and to competency. An LLM can draft them ("translate this project into a JD and a competency scorecard") as a starting point.

13.5 Implications for hiring and allocation

- **Articulation:** people must be able to articulate purpose and requirements. Where this is unclear in an interview, put the candidate on hold and record the reasons.
- **Self-awareness and humility:** prefer people who know where they stand over those who claim they can do anything.
- **Relay order:** people with deep comprehension run the first leg (purpose, scope, scorecard design); technically strong people who are still building depth follow in later legs.

Hiring stance: most companies must hire for competency because they cannot wait 12 to 18 months. With access to a large talent pool and time to develop people, we can hire for aptitude and capability, then build competency through rapid, real deliveries. What matters is that people know when they are operating from capability rather than competency.

14. Tool and product vision

The framework is intended to be embedded in how work is managed, not kept as a separate exercise. The minute someone clicks on a task in the project-management tool, the scorecard appears. It applies to everything the team does, technical and non-technical.

Capability	Description
Task-linked scorecard	Scorecard pops up when a task is opened; the ODTQRC must be completed and clarity-checked before work starts
KPI and guideline authoring	LLM-assisted discovery of KPIs; qualitative and quantitative guideline editor; reference scorecard templates
Self-appraisal	Private self-scoring against the same scorecard before submission
Judging	Expert or LLM judge scores the output against the guidelines
Gating and QTC	Blocks progression below target; tracks time and cost in real time; applies Q x T x C
RAG views	Colour-coded views by task, activity, milestone and project, and by individual, team and organisation
Red diagnosis and gaps	Classifies reds (skill, aptitude, will); estimates time to acquire a missing capability level
Analytics	Descriptive first, then predictive and prescriptive: recommends allocations and how teams can work together

14.1 The ODTQRC tool and the audit tool

Two tools carry the framework: the ODTQRC tool, which defines tasks with its clarity agent, and the audit tool, which checks the work and communication produced. When asked whether the scorecard belongs in these tools, the answer in the discussion was that it is part of everything the team does. Both tools share a centralized memory (the TGL memory database), so definitions, scorecards, scores and history are available to each other and are not re-entered.

Audit stage	Purpose (as drafted from notes)
1. Clarity checks	Is the content clear, unambiguous and complete for its purpose?
2. Actionability	Can someone act on it: owners, next steps and decisions explicit?
3. Parallel validation	Independent checks run in parallel (for example several judges or validators) so that no single judgement decides the score
4. Voice analysis	Extends the audit to spoken communication such as meetings and calls
5. Copilot	Guides the author as they work, so that an average communicator performs like an excellent one

Current emphasis for the tools: Quality and Time. Cost is tracked and will be enforced as the tools mature.

Design constraint: at around 2,400 scored tasks a day, the judge must be reliable. Scores that are wrong in either direction destroy trust in the system quickly.

Proposed: build the Quality Scorecard application as the first pilot of the Data-Driven Development Framework (see Section 16).

15. Reference scorecard library

The library starts small and grows continuously; the expectation is 200 to 250 or more scorecard types over time. Initial candidates raised in the discussion:

Work type	What quality means here (indicative)
Research outputs	Structured, sufficient and well-sourced; directly addresses the problem of limited or low-quality research
Requirements / specification document	Purpose, scope, unambiguity, completeness; target 10 for foundational specs
Deliverables (general)	Fitness for the stated purpose and scope
Meetings and conversations	Preparation, focus, outcomes; "are they well prepared, or all over the place?"
Minutes of meetings	Accuracy, completeness, actions and owners
Brainstorming sessions	Clarity of the question, breadth of ideas, convergence
Prompts for AI tools	Preparation and precision, so that tokens and time are not wasted
Cloud deployment and technical tasks	Purpose understood, correctness, repeatability
Assessment reports, proposals and client emails	Professional tone, clarity, no "kiddish" output; floor of 7
Understanding of a discussion	Captured understanding scored before implementation starts; gaps exposed by probing questions

Practice before creation: newcomers first apply established scorecards repeatedly. Creating new scorecards comes later, once understanding has matured (in Bloom's-taxonomy terms, understanding and applying before creating).

16. Pilot, roadmap and link to the Data-Driven Development Framework

This section is a drafting addition, not part of the discussion. The Quality Scorecard application is the Phase 1 learning sprint of the Data-Driven Development Framework v1.1, so this framework can serve as the Initial Product Scope for that pilot. The mapping below is proposed.

Initial Product Scope element	Content from this framework
Problem	Research and preparation are limited, unstructured or low quality; quality cannot be reviewed by hand at scale; work starts without objectives or specs; self-reported status is unreliable.
Actors	Task owner (self-appraisal); lead or manager (targets, allocation, diagnosis); judge (expert or LLM); scorecard designer; leadership (RAG views).
Expected outcomes	Every task scored against agreed KPIs; gates enforced; honest RAG at every level; reds diagnosed and acted on.
Constraints	Judge reliability at about 2,400 tasks a day; guidelines must be unambiguous; one shared memory across tools; internal use first; POC target 6 to 7.
Broad functional scope	ODTQRC task definition and clarity agent (Section 7); scorecard design (8 and 9); scoring and gating with QTC (6 and 10); self-appraisal (11); diagnosis and capability/competency (12 and 13); audit tool, shared memory, RAG views and analytics (14).

16.1 Proposed roadmap

Phase	Activities
1. Orientation	Share this framework with the core team; agree the scale, colour bands, targets and QTC rule.
2. Seed ODTQRC and scorecards	Adopt the ODTQRC template for all new tasks; core team designs the first reference scorecards (research, requirements document, meetings, prompts) using the seven steps.
3. Manual pilot	Apply the scorecards in a spreadsheet on live work, with self-appraisal first, then judging and gating.
4. Build the tools	Build the ODTQRC tool (with clarity agent) and the audit tool on a shared memory, through the Data-Driven Development Framework cycles, at a POC target of 7.
5. Capability and competency	Add role JDs (qualitative and quantitative), C-level descriptors and red diagnosis.
6. Scale	Extend to all products and teams; grow the reference library.
7. Analytics and evolution	Add predictive and prescriptive analytics; prepare for democratisation and external use.

Proposed pilot measures:

- Share of tasks reaching target on first submission
- Distribution of scores and colour bands by work type and team
- Agreement between LLM judge and expert judge on the same work
- Guideline revisions triggered by disputed or inconsistent scores
- Reds by diagnosed cause (skill, aptitude, will) and time to close capability gaps
- QTC outcomes: quality, time and cost misses tracked separately